GradeOps

Om Sharma | Aryan Basra | Ayush Kumar Singh

Contents

1 Introduction and Motivation

Grading handwritten examinations at scale is slow, inconsistent, and difficult to audit. A single mid-sized course can produce hundreds of scripts per assessment, and the marking that follows is repetitive yet demands careful, criterion-by-criterion judgement. Marks drift between graders, partial credit is applied unevenly, and the reasoning behind a score is rarely recorded in a form anyone can revisit later.

GradeOps is a platform that addresses this problem by combining modern vision and language models with a disciplined human review step. The system reads scanned handwritten answers, extracts their content, scores each answer against a structured rubric, and then routes every machine decision to a teaching assistant for confirmation before any grade becomes final. The intent is not to replace the human grader but to remove the mechanical labour around them: transcription, first-pass scoring, bookkeeping, and aggregation. The human remains the authority on the final mark.

This report documents the complete system in detail. It covers the user-facing application, the server and its data model, the authentication and authorization model, and, most importantly, the grading pipeline itself: how a page of handwriting becomes a transcribed answer, how that answer is scored against a rubric, how the machine's confidence and per-criterion reasoning are produced, and how a teaching assistant reviews and finalizes the result. Every claim here is drawn from the project's two source repositories and the running application.

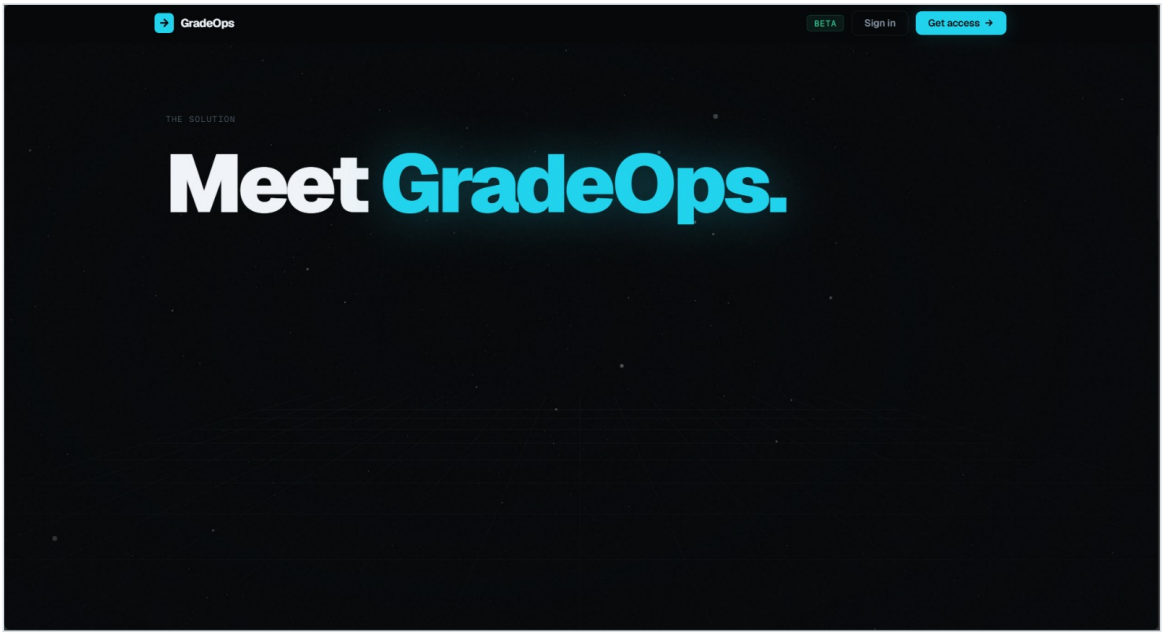

Figure 1: The GradeOps landing page. The product presents itself as a single grading workspace built around an automated pipeline with a human review stage.

GradeOps is split into two repositories. The frontend is a React application that provides the instructor and teaching-assistant dashboards. The backend is a FastAPI service that exposes the REST API, stores data, and runs the grading pipeline through an agentic workflow. The two communicate over HTTP, with the browser holding an authenticated session in a secure cookie. The remainder of this report works outward from this high-level picture into the specific logic of each layer.

2 System Overview

2.1 The end-to-end flow

At the highest level, a script travels through GradeOps along a fixed path. An instructor first defines an exam and its rubric, then uploads the student scripts as PDF files. The server stores each script, converts its pages into images, and starts a background grading job for it. That job reads the handwriting with a vision-language model, scores the transcribed answer against the rubric with a second model, and then deliberately pauses, holding its result and waiting for a human. A teaching assistant opens the review queue, inspects the machine's decision for each answer, and approves, overrides, or flags it. Once the human signs off, the job resumes, the final score is computed and stored, and the result becomes visible to the instructor for release.

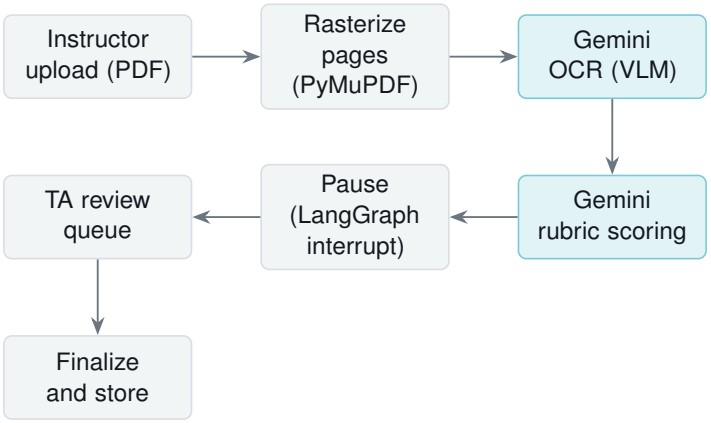

Figure 2: The grading lifecycle. The two shaded stages are AI model calls; the rest are deterministic server logic. The pause between scoring and review is what makes the workflow human-in-the-loop.

2.2 Why an agentic state machine

The defining design choice is that the grading job is modelled as a persistent state machine rather than a single linear function. Each script's progress is a graph with named stages, and the graph can pause mid-execution, save its entire state to disk, and resume later when a human supplies input. This is what allows a teaching assistant to step in at exactly the right moment, on their own schedule, without the server holding a request open or losing the machine's work. The mechanism is provided by LangGraph and is described in full in Section 6.

2.3 Repository structure

Repository	Responsibility
Frontend	React dashboards for instructors and teaching assistants, client routing, role guards, and the Axios client that carries the session cookie.
Backend	FastAPI REST API, PostgreSQL data layer, JWT cookie authentication, and the LangGraph grading pipeline with its vision and language model calls.

Table 1: The two halves of GradeOps and what each owns.

3 Technology Stack

The stack was chosen to keep the interface highly responsive while letting the backend offload heavy model computation to the background. The frontend leans on a single design-token system instead of a CSS framework, which keeps the dark, minimal aesthetic consistent across every screen.

Frontend	Role
React 19, Vite 8	Component framework and build tool.
React Router v7	Client-side routing with role-based route guards.
Axios	HTTP client, configured to send the session cookie on every request.
Framer Motion	Animation, using a small set of shared spring and easing presets.
Three.js (`@react-three/fiber`, `drei`)	The particle and grid visuals on the landing page.
Bootstrap, React-Bootstrap, react-pro-sidebar	Selected UI components and the responsive sidebar.
Inline styles + tokens	All styling derives from a single `tokens.js` source of truth.

Table 2: Frontend technologies.

Backend	Role
FastAPI, Python	REST API and asynchronous background task runner.
LangGraph	The grading state machine, checkpointed to SQLite.
Google Gemini	Both vision-language OCR of handwriting (`gemini-3.1-flash-lite`) and rubric reasoning and scoring (`gemini-3.1-flash-lite`).
PostgreSQL + SQLAlchemy	Application data and ORM.
SQLite	LangGraph checkpoint store (`checkpoints.db`).
PyJWT, pwdlib (argon2)	Token signing and password hashing.
PyMuPDF	Reading and rasterizing uploaded exam PDFs.

Table 3: Backend technologies.

4 Frontend Architecture

4.1 Routing and role-based access

Routing is handled entirely on the client. The application defines a guard component that reads the current user from context and compares their role against the set of roles a route permits. A visitor without a session is redirected to the login page, and a logged-in user who tries to reach a portal that does not belong to their role is turned away. The same enforcement is repeated on the server, so the client guard is a usability layer rather than the real security boundary.

Path	Access	View
`/`	Public	Landing page
`/login`, `/register`	Public	Authentication
`/dashboard`	Authenticated	Role router
`/instructor`, `/instructor/exams`, `/instructor/grades`	Instructor	Instructor portal
`/ta` and its sub-routes	Teaching assistant	TA review portal

Table 4: Client routes and the role each requires.

4.2 Authentication and session state

The Axios instance is configured once with the backend base URL and with credentials enabled, so the browser attaches the session cookie automatically to every call. A React context holds the current user along with login and logout helpers. When the application loads, the context asks the server who the user is by calling the session endpoint, and rehydrates from that response. No token is ever stored in JavaScript-accessible storage; the credential lives only in an HTTP-only cookie, which is unreadable to page scripts and therefore resistant to token theft

through cross-site scripting.

4.3 Design system

Every colour, surface, and motion curve in the application comes from one tokens module. The theme is a near-black background with two accent colours, cyan and emerald, reserved for primary actions and positive states. Three animation presets, a tight spring, a looser spring, and an exponential ease, are imported wherever motion is needed, which keeps transitions feeling like one coherent system. Typography pairs a humanist sans for interface text with a monospace face for identifiers, codes, and metrics.

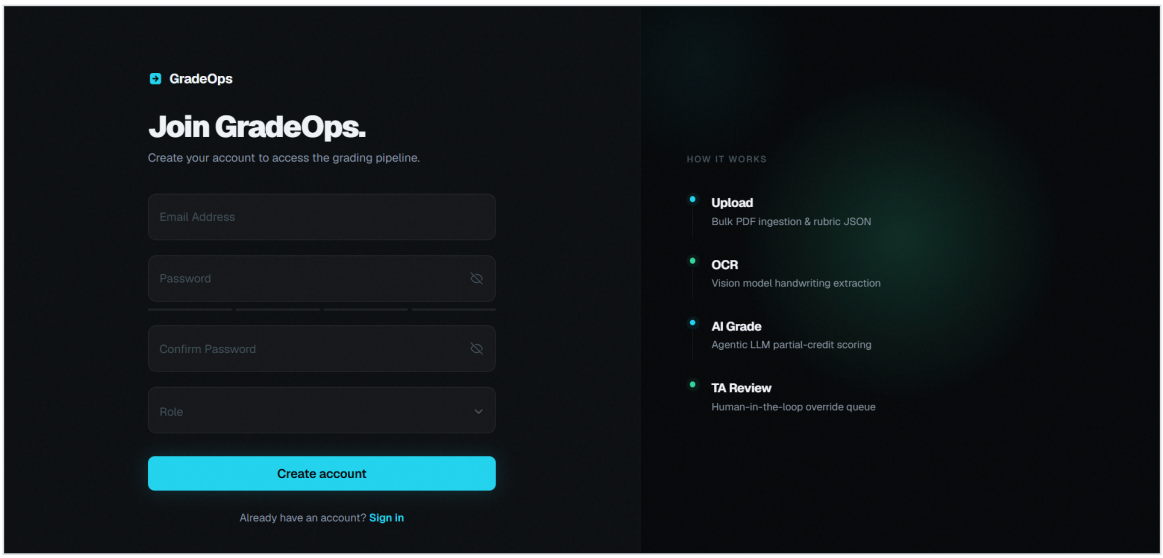

Figure 3: The registration screen. The right panel doubles as a summary of the pipeline: bulk upload and rubric JSON, vision-model OCR, agentic partial-credit scoring, and the human-in-the-loop review queue.

4.4 Component library and hooks

A shared module provides the reusable building blocks: a magnetic primary button with several variants, a floating-label input with a focus glow, a custom animated select, and small primitives such as a loading spinner and a status dot. A handful of custom hooks support the experience, including an animated counter for the instructor's statistic cards, a breakpoint hook that reports mobile and tablet sizes from a resize listener, and a scroll-trigger hook used for reveal animations on the landing page. The sidebar adapts to screen size, showing a full labelled rail on desktop, a compact icon rail on tablet, and a drawer on mobile, and it animates its active item with a sliding pill.

4.5 The instructor portal

The instructor portal opens on an overview that summarises the workspace: total exams, scripts graded, pending review, and flagged counts, followed by a table of recent exams and their status.

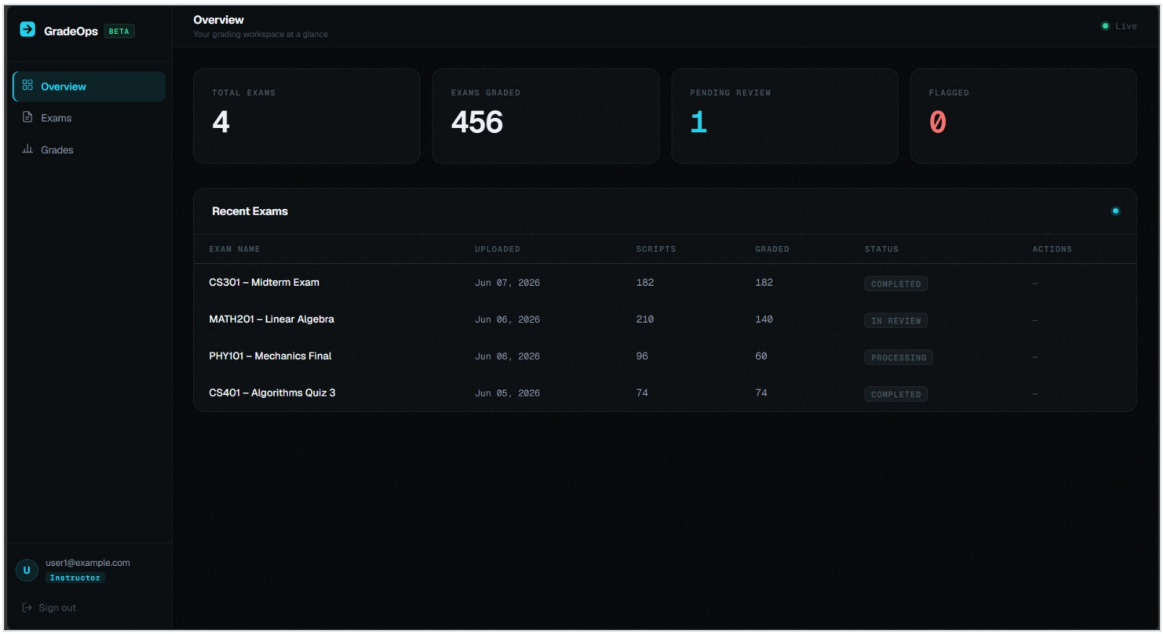

Figure 4: Instructor overview. The headline statistics are animated counters, and the recent-exams table mirrors the per-exam status the backend computes from submission counts.

The exams screen is where an instructor defines and feeds the pipeline. A rubric is written as structured JSON, validated in place, and the student scripts are added through a drag-and-drop area that accepts several PDFs at once. The right rail lists the uploaded batches with their progress.

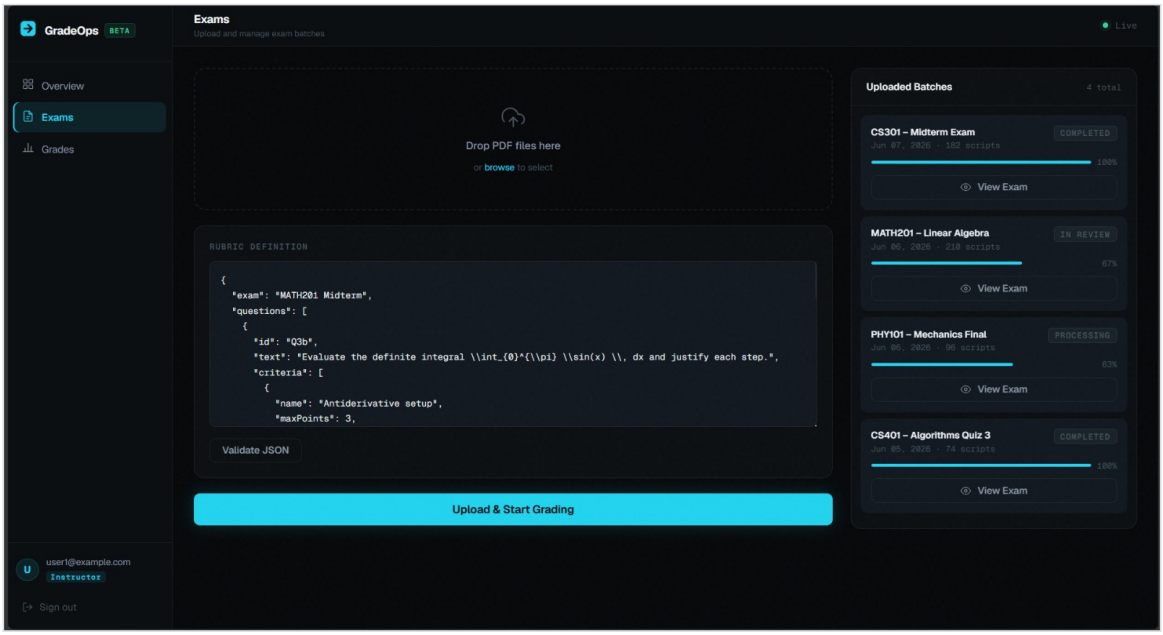

Figure 5: The exams screen. The rubric is authored as JSON with an explicit list of criteria per question, and the upload area submits a batch of PDFs that the backend turns into grading jobs.

The grades screen aggregates every student's machine score against the maximum derived from the rubric, shows a percentage and status, and lets the instructor release results.

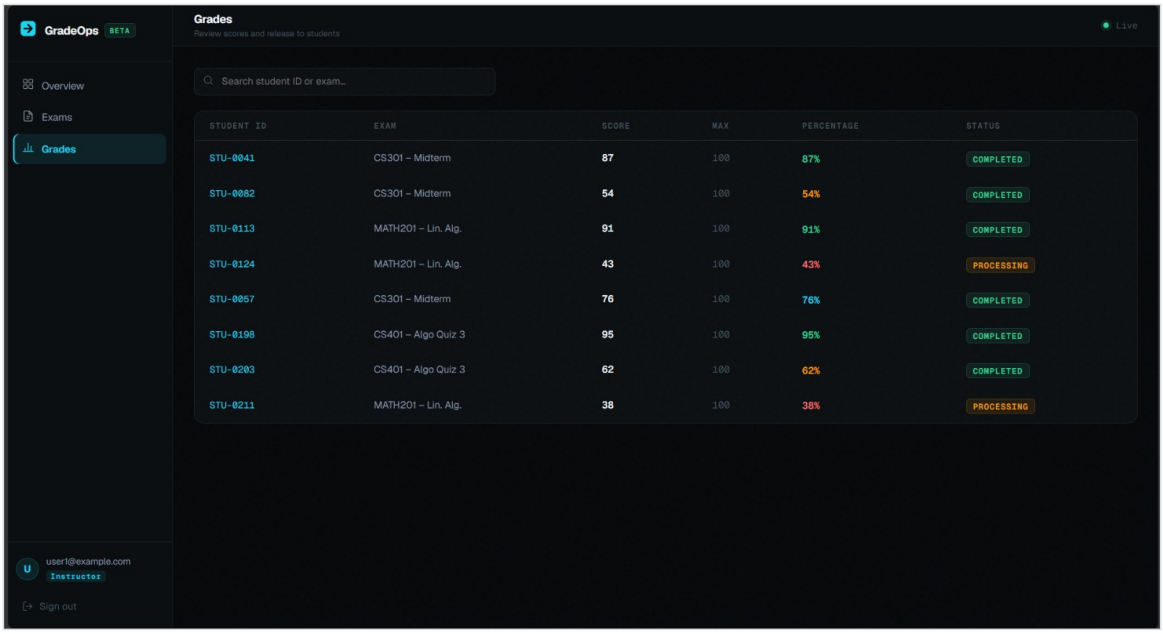

Figure 6: The grades table. Each score is shown against the maximum the backend computes by summing the rubric's per-criterion points, with a release control for the instructor.

4.6 The teaching-assistant portal

A teaching assistant lands on a list of exams that still have scripts awaiting review. When there is nothing to grade, the screen shows a clear empty state rather than placeholder content.

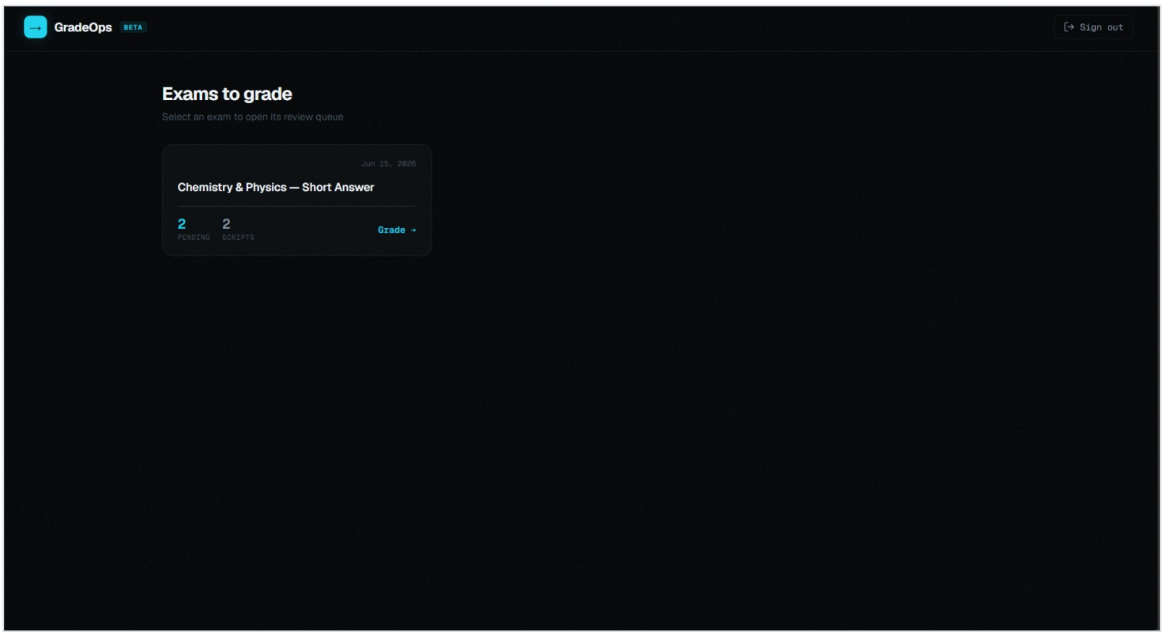

Figure 7: The TA landing page. It lists only exams with pending reviews; here the queue is genuinely empty, so the interface says so plainly.

Selecting an exam opens its review queue. The left column lists the scripts and their machine confidence; the main panel shows everything a reviewer needs for one answer: the question and its rubric, the extracted handwriting with an OCR confidence reading, and the machine's

score, confidence, and written justification.

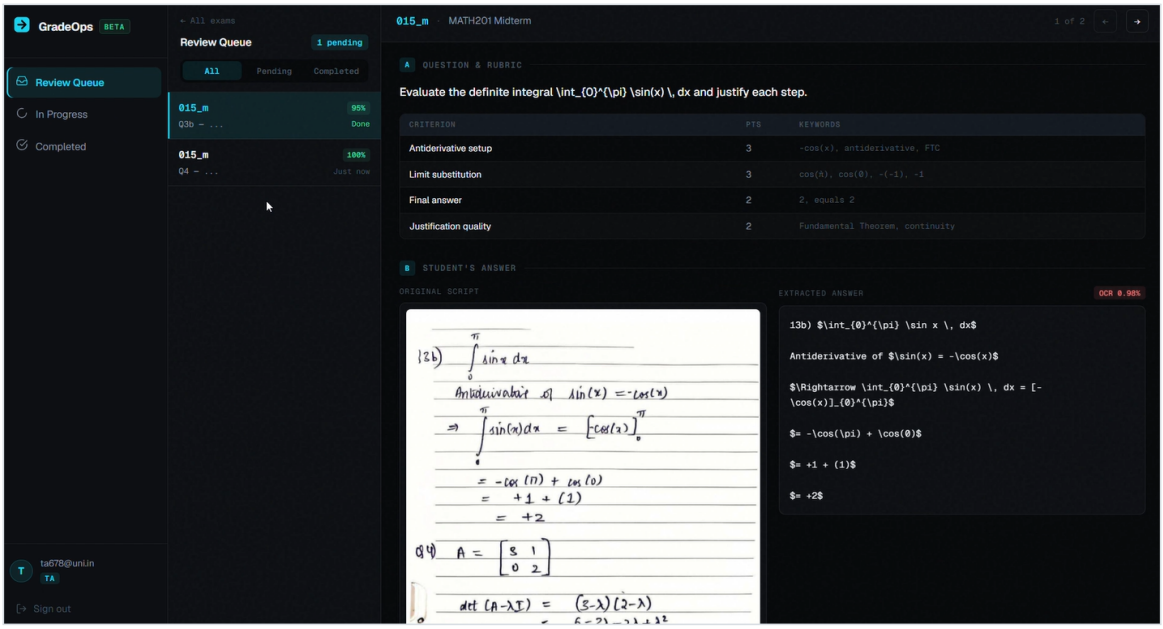

Figure 8: The review queue. For one answer the reviewer sees the question and rubric, the transcribed student work with an OCR confidence figure, and the AI's awarded score and confidence.

Scrolling reveals the per-criterion breakdown and the decision controls. Each rubric criterion is marked full, partial, or none, with the points awarded against its maximum. The reviewer then approves or overrides with a required reason, and can drive the whole flow from the keyboard.

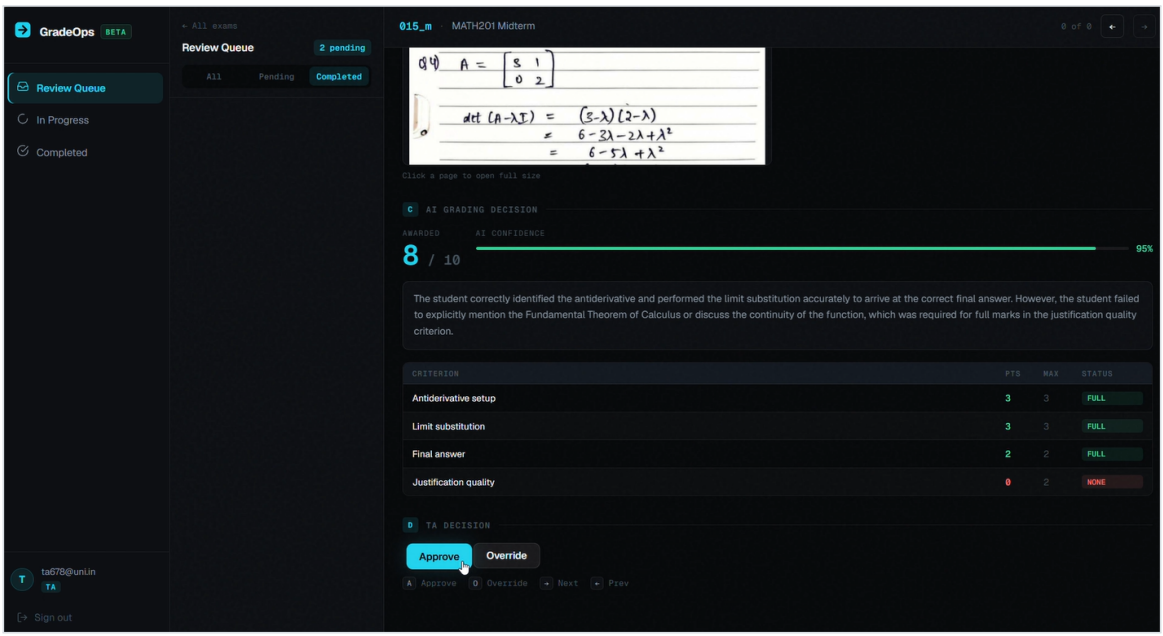

Figure 9: The per-criterion match and the decision controls. The status of each criterion and the keyboard shortcuts make review fast and consistent.

4.7 Plagiarism and Similarity Detection

GradeOps incorporates a cross-submission similarity detection layer that surfaces potential academic dishonesty directly inside the TA review queue. Rather than relying on naive string matching or overloading the grading model with context, the system employs a hybrid, asynchronous Map-Reduce architecture combining mathematical vector embeddings with a secondary LLM verification pass.

4.7.1 Detection mechanism

Because plagiarism detection is inherently an aggregate operation—a paper cannot be compared against a classroom of papers that have not yet been processed—the similarity check does not run during the individual grading nodes. Instead, it is triggered automatically by a batch completion hook. When the final script in an exam batch finishes its grading pipeline, the server fires an asynchronous background task to evaluate the entire cohort simultaneously.

The detection operates in two phases:

1. **Vector Similarity (The Net):** The clean LaTeX transcriptions of every student's answer are converted into mathematical vectors using Google's `text-embedding-004` model. The backend calculates a cosine similarity matrix in-memory using `scikit-learn`. Any pair of answers with a semantic similarity score exceeding 85% is flagged for the next phase.
2. **LLM Verification (The Filter):** To prevent false positives on standard mathematical proofs or multiple-choice questions where identical text is expected, the highly similar pairs are passed to a secondary model (`gemini-1.5-flash`). This evaluator is bound to a strict structured output schema and explicitly instructed to distinguish between coincidental alignment (e.g., standard arithmetic) and suspicious collusion.

4.7.2 What the flag captures

The similarity detection is designed to catch structural collusion rather than coincidental agreement on standard steps. Two submissions that both write $-\cos(x)$ as the antiderivative of $\sin(x)$ are not flagged, because that is the correct answer and independent students will produce it. Two submissions that share an unusual algebraic shortcut, an identical mid-step error that happens to cancel, or identical prose in their justification paragraphs are flagged, because that pattern is unlikely to arise independently.

The model records a brief natural-language reason alongside the list of similar roll numbers, which gives the reviewer enough context to judge whether escalation is warranted without requiring them to re-read both scripts from scratch.

4.7.3 Queue injection and the data model

To maintain high performance and avoid database lockouts during batch processing, the similarity engine does not mutate the core PostgreSQL relational database. Instead, the final verified flags are compiled into a flat JSON dictionary (`plagiarism.json`) and saved directly into the exam's physical upload directory.

When a teaching assistant opens the review queue, the FastAPI endpoint dynamically merges this JSON file with the interrupted LangGraph state. The result is encoded in the `similarityFlag` field of the `QuestionGradingResult` schema:

Listing 1: The similarity flag schema dynamically injected into the review queue.

```
class SimilarityFlagData(BaseModel):
    count: int                                  # Number of similar papers found
    papers: list[str]                           # Roll numbers of similar
    submissions

class QuestionGradingResult(BaseModel):
    ...
    similarityFlag: Optional[SimilarityFlagData] = None
```

When the field is populated, the submission is tagged for elevated review. The TA queue highlights these scripts visually, and the interface shows exactly which other student IDs share the flagged pattern, allowing the reviewer to inspect both scripts side by side before finalizing the grade.

5 Backend Architecture

5.1 The application and its data model

The backend is a FastAPI application backed by PostgreSQL through SQLAlchemy. Three tables carry the entire domain. Users hold an email, a hashed password, and a role. Exams hold a title, a rubric stored as JSON, the owning instructor, an upload timestamp, and a status. Submissions hold a student roll number, the paths to the stored PDF and its rasterized page images, the machine score and justification once produced, a status, and the exam they belong to. Deletions cascade from an exam to its submissions and from a user to their exams, so removing an exam cleans up everything beneath it.

Listing 2: The three core tables, abbreviated from the SQLAlchemy models.

```
class User(Base):
    id = Column(Integer, primary_key=True)
    email = Column(String, unique=True, nullable=False)
    password = Column(String, nullable=False)    # argon2 hash
    role = Column(String, nullable=False)        # "instructor" | "ta"

class Exam(Base):
    id = Column(Integer, primary_key=True)
    title = Column(String, nullable=False)
    rubric = Column(JSON, nullable=False)
    instructor_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"))
    uploaded = Column(DateTime(timezone=True), server_default=func.now())
    status = Column(String, server_default="Processing")

class Submission(Base):
    id = Column(Integer, primary_key=True)
    student_roll_no = Column(String, nullable=False)
    pdf_path = Column(String, nullable=False)
    images_path = Column(String, nullable=False)
    ai_score = Column(Integer, nullable=True)
```

```
    ai_justification = Column(String, nullable=True)
    status = Column(String, server_default="processing")
    exam_id = Column(Integer, ForeignKey("exams.id", ondelete="CASCADE"))
```

5.2 The submission status lifecycle

A submission moves through a small, well-defined set of states. The status is the single source of truth for where a script is in the pipeline and is what the queue and grades endpoints filter on.

Status	Meaning
`processing`	Uploaded; the grading job is running OCR and scoring.
`pending_review`	The machine has scored and the job has paused for a human.
`completed`	A teaching assistant has finalized the result; the score is stored.
`error`	The pipeline failed or did not reach the review pause.

Table 5: The submission status lifecycle.

5.3 Authentication internals

Authentication is token-based, but the token is delivered through a cookie rather than a header. On login the server verifies the password and issues a JSON Web Token signed with the HS256 algorithm using a server secret, carrying the user's identity and an expiry. That token is set as an HTTP-only cookie. On every protected request, a dependency reads the cookie, decodes and verifies the token, and loads the corresponding user. Passwords are never stored in the clear; they are hashed with argon2 through the pwdlib library, and a dummy hash is kept so that the verification path takes a similar amount of time whether or not the email exists, which avoids leaking account existence through timing.

Listing 3: Issuing and verifying the session token.

```
def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + timedelta(
        minutes=int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES")))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, os.getenv("SECRET_KEY"), algorithm="HS256")

async def get_current_user(request: Request, db: Session = Depends(get_db)):
    token = request.cookies.get("access_token")
    if token is None:
        raise HTTPException(401, "Not authenticated")
    decoded = jwt.decode(token, os.getenv("SECRET_KEY"), algorithms=["HS256"])
    user = db.query(models.User).filter(models.User.id == decoded.get("id")).
    first()
    if not user:
        raise HTTPException(401, "Invalid credentials!")
    return user
```

6 The Grading Pipeline

This section is the core of the report. It follows a single script from the moment it is uploaded to the moment the machine pauses for review, explaining each stage and the precise logic it runs.

6.1 Ingestion and rasterization

The upload endpoint accepts a batch of PDF files together with the target exam, and it is restricted to instructors. For each file, the server derives the student identifier from the filename, removing any unsafe characters first. If a submission already exists for that student on that exam, the old record and its files are deleted so that re-uploading cleanly replaces a previous attempt. A new submission row is created with the status `processing`, the PDF is written to disk, and PyMuPDF opens the document and renders each page to a PNG image at 150 dots per inch. Finally, a background task is scheduled to grade the submission, so the HTTP response returns immediately while the heavy work happens asynchronously.

Listing 4: Rasterizing each page and scheduling the async grading job.

```
with pymupdf.open(pdf_destination) as doc:
    for page in doc:
        pix = page.get_pixmap(dpi=150)
        pix.save(f"{imgs_destination}/page-{page.number}.png")

background_tasks.add_task(execute_grading_pipeline, current_sub_id)
```

6.2 The grading state machine

The grading job itself is a LangGraph state machine. Its state is a typed dictionary that travels through the graph, accumulating fields as each stage runs. The graph has three nodes wired in a straight line: OCR, then scoring, then a human-intervention stage, after which it ends.

Listing 5: The grading state and the graph wiring.

```
class GradingState(TypedDict):
    submission_id: int
    student_id: str
    page_img_paths: list[str]
    rubric: dict[str, Any]
    ocr_text: str
    detailed_analysis: dict
    status: str
    feedback: str

builder = StateGraph(GradingState)
builder.add_node("gemini_ocr", run_gemini_ocr_node)
builder.add_node("gemini_grader", grade_with_gemini_model)
builder.add_node("human_intervention", human_intervention_decider)
builder.set_entry_point("gemini_ocr")
builder.add_edge("gemini_ocr", "gemini_grader")
builder.add_edge("gemini_grader", "human_intervention")
builder.add_edge("human_intervention", END)
```

The graph is compiled dynamically using an asynchronous checkpointer (`AsyncSqliteSaver`) backed by `aiosqlite`. Every exam script is executed concurrently under a thread identifier derived from its submission, of the form `submission_{id}`. This identifier is what allows the system to accurately retrieve a paused job later: when a teaching assistant is ready to review a script, the FastAPI backend asynchronously fetches the graph state (`aget_state`) for that thread to read the specific UI payload the machine paused on. Because the checkpointer persists this state directly to a local database file, the paused human-in-the-loop review is entirely non-blocking and safely survives both API timeouts and server restarts.

6.3 Node one: handwriting recognition with Gemini

The first node performs optical character recognition using Gemini's vision-language capabilities. To prevent server timeouts on lengthy exams, the node processes page images concurrently rather than sequentially, using an asynchronous semaphore to regulate the requests and avoid rate limits. For each page, the node reads the file, encodes it as base64, and sends it to the Gemini model together with a tightly written instruction.

The instruction asks the model to act as an academic digitizer: to correct obvious cursive misreadings without altering the student's logic, to render all mathematics and scientific notation in LaTeX, to preserve question labels so the document structure survives, to ignore scribbles, crossed-out text, and page numbers, and to return only the transcription with no conversational text. The concurrently gathered transcriptions are concatenated into a single block, which becomes the `ocr_text` field of the state.

Listing 6: The OCR node, condensed. An asyncio semaphore regulates concurrent processing of the exam pages.

```python
gemini_vlm = ChatGoogleGenerativeAI(model="gemini-3.1-flash-lite", temperature=0.0)

async def run_gemini_ocr_node(state):
    sem = asyncio.Semaphore(3) # Regulates concurrent API requests

    async def process_single_page(index, img_path):
        async with sem:
            with open(img_path, "rb") as img_file:
                b64_img = base64.b64encode(img_file.read()).decode("utf-8")
            message = HumanMessage(content=[
                {"type": "text", "text": DIGITIZER_INSTRUCTION},
                {"type": "image_url", "image_url": {"url": f"{b64_img}"}}])

            response = await gemini_vlm.ainvoke([message])
            return f"\n\n--- Page {index+1} ---\n\n" + response.content

    # Fire all page requests concurrently
    tasks = [process_single_page(i, path) for i, path in enumerate(state["page_img_paths"])]
    results = await asyncio.gather(*tasks)

    return {"ocr_text": "".join(results)}
```

The choice to transcribe rather than grade directly from the image is deliberate. It produces an auditable text artefact that the teaching assistant can read and check against the original, and

it cleanly separates the perception problem, reading messy handwriting, from the judgement problem, applying a rubric. The instruction's insistence on LaTeX for mathematics is what allows an integral or a complexity bound to survive into the review interface in a readable form. Using Gemini for both OCR and scoring also simplifies the integration surface: a single API key, a single provider, and a consistent structured-output contract across both pipeline stages.

6.4 Node two: rubric scoring with Gemini

The second node performs the actual grading. It uses Gemini at temperature zero, configured to return a structured object rather than free text. The prompt casts the model as an expert teaching assistant and gives it the student identifier, the rubric serialized as JSON, and the transcribed answer. It instructs the model to evaluate the answer against each rubric criterion, decide the points awarded for that criterion out of its maximum, compute a status from that comparison, sum the awarded points into a total score, and write a single cohesive justification explaining the deductions.

The scoring rule for each criterion is explicit and mechanical:

- **full** when the awarded points equal the maximum,
- **partial** when the awarded points are above zero but below the maximum,
- **none** when no points are awarded.

The overall score for a question is defined as the sum of the awarded points across its criteria, so the granular breakdown and the headline number can never disagree.

Listing 7: The scoring node, condensed to its essentials.

```
gemini_model = ChatGoogleGenerativeAI(model="gemini-3.1-flash-lite", temperature
    =0.0)
structured_gemini = gemini_model.with_structured_output(FullExamGradingResult)
grading_chain = prompt | structured_gemini

def grade_with_gemini_model(state):
    result = grading_chain.invoke({
        "rubric": json.dumps(state["rubric"]),
        "student_id": state["student_id"],
        "ocr_text": state["ocr_text"]})

    return {"detailed_analysis": result.model_dump(), "status": "
    pipeline_complete"}
```

Because the model is bound to a structured output schema, its response is forced into a fixed shape that the frontend can render directly. The schema is the contract between the grader and the review interface, and it is worth stating in full.

Listing 8: The structured grading schema the model must return.

```
class RubricCriterionMatch(BaseModel):
    name: str
    awarded: int
    max: int
    status: Literal["full", "partial", "none"]

```

```python
class QuestionGradingResult(BaseModel):
    id: str                        # UI card id, e.g. "q1"
    studentId: str
    questionRef: str               # rubric question id, e.g. "Q3b"
    questionText: str
    rubric: list[RubricCriterion]
    ocrText: str
    ocrConfidence: float = 95.0
    aiScore: int                   # sum of awarded points
    maxScore: int
    aiJustification: str
    rubricMatch: list[RubricCriterionMatch]
    confidence: int                # 0-100, AI confidence in the decision
    status: str = "pending"
    timeAgo: str = "Just now"
    similarityFlag: Optional[SimilarityFlagData] = None

class FullExamGradingResult(BaseModel):
    questions: list[QuestionGradingResult]
```

Each graded question therefore carries not just a number but the full reasoning around it: the transcribed answer that was judged, a per-criterion breakdown with a status for each, a holistic justification in words, an estimate of OCR confidence, and a separate confidence in the grading decision itself. The optional similarity flag records other student identifiers whose logic resembles this answer, which is what the flagged view surfaces.

6.5 Node three: pausing for the human

The third node is what turns an automated grader into a human-in-the-loop system. It does almost nothing except stop. It takes the structured analysis produced by the scoring node and raises a LangGraph interrupt carrying that payload. An interrupt suspends the graph and persists its state through the checkpointer; the job is now frozen, holding the machine's proposed grades, until something resumes it.

Listing 9: The human-intervention node simply interrupts with the AI payload.

```python
def human_intervention_decider(state):
    ui_payload = state.get("detailed_analysis", {})
    human_review = interrupt(ui_payload)          # pauses here
    return {"status": "approved",
            "detailed_analysis": {"questions": human_review.get("questions", [])
    }}
```

When the background job returns, the server inspects the graph state. If the job paused at an interrupt, the submission is marked `pending_review` and becomes visible in the teaching assistant's queue. If it did not reach the pause, the submission is marked `error`. This check is the bridge between the in-memory graph and the durable database status.

6.6 The batch completion hook

Because the grading pipeline operates concurrently across dozens of background tasks, the system requires a mechanism to know when an entire exam batch is complete so it can safely execute aggregate operations.

This is achieved through a batch completion hook at the end of the pipeline execution function. After a graph successfully hits the interrupt pause and its PostgreSQL status is updated to `pending_review`, the database is queried to count how many submissions for that specific exam are still marked as `processing`. If the count is exactly zero, the current thread recognizes it is the final script in the batch. It immediately spawns a new, detached asynchronous task (`asyncio.create_task`) to execute the cross-submission similarity detection matrix, ensuring the plagiarism check runs instantly and automatically without requiring a manual trigger from the instructor.

7 Human-in-the-Loop Review

7.1 Assembling the queue

When a teaching assistant opens an exam, the server builds the review queue by finding every submission for that exam whose status is `pending_review`. For each, it looks up the paused graph state by thread identifier and reads the interrupted payload, which is exactly the structured analysis the scoring node produced. Each question in that payload is tagged with its submission identifier and added to a single combined queue. The reviewer therefore sees a flat list of answers to judge, while the system retains the link back to the submission each one belongs to.

Listing 10: Reading paused machine decisions back out of the graph.

```
for sub in pending_subs:
    config = {"configurable": {"thread_id": f"submission_{sub.id}"}}
    paused_state = graph.get_state(config)
    if paused_state.tasks and paused_state.tasks[0].interrupts:
        ui_payload = paused_state.tasks[0].interrupts[0].value
        for q in ui_payload.get("questions", []):
            q["submission_id"] = sub.id
            master_queue.append(q)
return {"queue": master_queue}
```

7.2 Making a decision

For each answer the reviewer has three choices. Approve accepts the machine's score as proposed. Override opens a small editor where the reviewer adjusts the score and must supply a reason, which is recorded against the answer. Flag marks the script for senior review. The interface is built to be driven from the keyboard, with single keys for approve, override, and flag and the arrow keys to move between answers, so a reviewer can work through a queue quickly without leaving the home row.

The interface tracks session progress as the reviewer works. When every question belonging to a given submission has been decided, whether approved, overridden, or flagged, the frontend finalizes that submission by sending the collected decisions back to the server. This grouping by submission is why a student's whole script is finalized as a unit rather than one answer at a time.

7.3 Resuming and finalizing

Finalization resumes the paused graph. The server issues a resume command carrying the reviewer's decisions, which unfreezes the human-intervention node and lets the graph run to completion. From the final state the server sums the awarded points across all questions into a total score, stores that score and the full decision payload on the submission, and marks it `completed`. The result now appears on the instructor's grades screen.

Listing 11: Resuming the graph and persisting the final score.

```
final_state = graph.invoke(Command(resume=payload.model_dump()), config=config)
total_score = sum(int(q.get("aiScore", 0))
                  for q in final_state.get("detailed_analysis", {}).get("
    questions", []))
submission.ai_score = total_score
submission.ai_justification = json.dumps(final_state["detailed_analysis"])
submission.status = "completed"
db.commit()
```

8 Grading Logic in Detail: A Worked Example

To make the grading concrete, consider the answer shown in the review screenshots. The question asks the student to evaluate the definite integral $\int_0^{\pi} \sin(x)\,dx$ and justify each step. The rubric for this question carries four criteria worth ten points in total: antiderivative setup (3), limit substitution (3), final answer (2), and justification quality (2).

The transcribed answer shows the student writing the antiderivative as $-\cos(x)$, substituting the limits to obtain $-\cos(\pi) - (-\cos(0))$, simplifying to $-(-1) - (-1) = 1 + 1$, and concluding the value is 2, with a closing sentence invoking the Fundamental Theorem of Calculus. The OCR confidence for this transcription is reported at roughly ninety-four percent.

The model awards full marks on the first three criteria and a partial mark on the fourth, because the justification omits an explicit mention of the continuity requirement that the Fundamental Theorem needs. The resulting breakdown is shown below.

Criterion	Awarded	Max	Status
Antiderivative setup	3	3	full
Limit substitution	3	3	full
Final answer	2	2	full
Justification quality	1	2	partial
Total	**9**	**10**	

Table 6: The per-criterion result for the worked example. The total of nine is exactly the sum of the awarded column, and the grading confidence is reported at ninety-one percent.

This example illustrates the whole logic in miniature. The machine read the handwriting, judged each criterion independently, derived the total by summation, attached a confidence to the

decision, and wrote a justification naming the specific deduction. A teaching assistant can read the transcription, agree or disagree with the single partial mark, and either approve the nine or override it with a reason, all in a few seconds.

9 Security Model

Security rests on a few deliberate choices. Authorization is enforced on the server, not merely in the interface: every protected endpoint resolves the current user from the session cookie, and sensitive actions additionally check that the user's role is correct and, where ownership matters, that the exam belongs to the requesting instructor. The session credential is a signed token delivered in an HTTP-only cookie, which page scripts cannot read, so a cross-site scripting flaw cannot exfiltrate it. Passwords are stored only as argon2 hashes. The signing secret and database credentials are read from the environment rather than committed to the repository. Cross-origin requests are restricted to the known frontend origin with credentials enabled, and the cookie's secure attribute is environment-driven so that it can be relaxed for plain-HTTP local development and enforced under HTTPS in production.

10 Data and State Persistence

GradeOps keeps two distinct stores for two distinct kinds of state, and the separation is intentional. PostgreSQL holds the durable relational record: users, exams, rubrics, and submissions with their final scores. SQLite holds the LangGraph checkpoints: the in-flight state of every grading job, including the paused payload a teaching assistant will review. The relational database answers questions about what the grades are; the checkpoint store answers questions about where each job is. Because the checkpoint store is written to disk, a paused review is not lost when the server restarts, and the queue can always be rebuilt by reading the persisted interrupt values for submissions still marked pending.

11 API Reference

Endpoint	Purpose
`POST /login/`	Authenticate and set the session cookie.
`POST /register/`	Create an account.
`GET /users/me`	Restore the current session and user.
`GET /users/logout`	Clear the session cookie.
`POST /exams/`	Create an exam from a title and rubric.
`GET /exams/`	List the instructor's exams with counts and status.
`GET /exams/to-grade`	Exams that have scripts awaiting review.
`GET /exams/:id/queue`	The review queue for one exam.
`POST /upload/`	Upload a batch of student PDFs for an exam.
`GET /upload/:id/review`	Read the paused machine decision for a submission.
`POST /upload/:id/review`	Finalize a reviewed submission.
`GET /upload/instructor_exams`	Aggregated grades for the instructor.

Table 7: The REST surface consumed by the frontend.

12 Limitations and Future Work

The system is a working prototype, and while the architecture successfully decouples the similarity engine from the state machine by extracting transcriptions directly from the relational database, several areas are candidates for hardening before a large-scale deployment.

First, the plagiarism detection currently computes an $O(N^2)$ cosine similarity matrix in-memory using `scikit-learn`. While this Map-Reduce approach is lightning-fast for typical university cohort sizes (e.g., hundreds of scripts), it would encounter memory and compute bottlenecks if scaled to massive open online courses (MOOCs) with tens of thousands of submissions.

Second, storing the final similarity flags in a local `plagiarism.json` file ties the architecture to a single server. In a distributed cloud environment where multiple worker nodes sit behind a load balancer, the server that writes the file might not be the server the teaching assistant connects to.

Moving this system from a single-server prototype to a horizontally scaled cloud production environment would require three structural shifts:

1. Migrating the in-memory similarity matrix to a persistent vector database (such as PostgreSQL with the `pgvector` extension) to enable high-speed Approximate Nearest Neighbor (ANN) searches.
2. Migrating the JSON file storage directly into the PostgreSQL relational schema to ensure global availability across all UI instances.
3. Introducing a centralized message broker (such as Redis) to safely coordinate the batch completion hooks across multiple concurrent worker nodes.

Finally, there is no relationship in the data model assigning teaching assistants to particular

exams, so any authenticated assistant can review any pending exam; introducing such a relationship would allow for scoped review queues. None of these limitations affect the core accuracy of the grading flow; they represent the natural evolutionary steps for scaling the platform.

13 Conclusion

GradeOps demonstrates a clean division of labour between machine and human. A vision-language model turns handwriting into auditable text, a language model scores that text against an explicit rubric and explains itself, and a state machine holds the result until a human confirms it. The interface around this pipeline is built for speed and clarity, and the server enforces a coherent security and data model beneath it. The result is a grading workflow that removes the mechanical burden of marking at scale while keeping a person firmly in control of every final grade.

A Appendix: Rubric JSON Schema

A rubric is authored as JSON with a top-level exam name and an array of questions. Each question carries an identifier, its text, and an array of criteria, where each criterion names what it measures, its maximum points, and optional keywords the grader can look for.

Listing 12: A rubric in the format the exams screen accepts.

```
{
  "exam": "MATH201 Midterm",
  "questions": [
    {
      "id": "Q3b",
      "text": "Evaluate the definite integral of sin(x) from 0 to pi and justify each step.",
      "criteria": [
        { "name": "Antiderivative setup", "maxPoints": 3, "keywords": ["-cos(x)", "FTC"] },
        { "name": "Limit substitution",  "maxPoints": 3, "keywords": ["cos(pi)", "cos(0)"] },
        { "name": "Final answer",        "maxPoints": 2, "keywords": ["2", "equals 2"] },
        { "name": "Justification quality","maxPoints": 2, "keywords": ["continuity"] }
      ]
    }
  ]
}
```

The maximum score for an exam is computed by summing the maximum points of every criterion across every question, which is the figure the grades screen shows each student's score against.

B Appendix: Environment and Setup

The backend reads its configuration from environment variables: the PostgreSQL connection string, the token signing secret, the token lifetime in minutes, and a flag controlling the secure attribute on the session cookie. For local development over plain HTTP this flag is set to false so the cookie is stored and returned; under HTTPS in production it is set to true. The frontend points its Axios client at the backend base URL with credentials enabled, and is served on the `localhost` origin so that it matches the backend's cross-origin policy and the cookie's origin.